



中山大學
SUN YAT-SEN UNIVERSITY

操作系统实验报告

实验五：fork、阻塞与并发

姓 名： 郭盈盈
学 号： 24312063
教学班号： 吴岸聪老师班
专 业： 保密管理
院 系： 计算机学院

2025 学年第二学期

一. 实验目的

1. 理解 `fork` 系统调用的语义，掌握父子进程上下文、栈空间、返回值和进程关系的维护方法。
2. 理解阻塞与唤醒机制，使用等待队列避免轮询带来的 CPU 浪费。
3. 理解竞态条件产生的原因，掌握自旋锁和信号量两种同步机制的实现与使用。
4. 在用户程序中使用 `fork` 创建并发执行流，并通过同步原语保护共享资源。
5. 完成多线程计数器、消息队列、哲学家就餐问题和 `fish` 同步输出程序，验证并发控制的正确性。

二. 实验环境

项目	配置
操作系统	YatSenOS / YYOS
开发语言	Rust (<code>no_std</code> 、 <code>no_main</code>)
目标架构	x86_64
引导环境	UEFI + OVMF
虚拟机	QEMU
报告工具	Typst

三. 实验内容概述

本次实验围绕“fork、阻塞与并发”展开。根据实验文档要求，`fork` 需要创建子进程，父进程返回子进程 PID，子进程返回 0；同时本实验中的 `fork` 不复制完整地址空间，而是共享代码段、数据段、堆和 bss 段，只为子进程提供独立的寄存器上下文和栈空间。因此，父子进程可以看到共享全局变量的修改，但各自的局部栈变量互不影响。

实验还要求实现进程阻塞和唤醒。对于等待某个事件的进程，若事件尚未发生，则将其放入等待队列并切换到其他就绪进程；当事件发生时，再把等待进程唤醒并放回就绪队列。信号量在此基础上实现 `new` 、 `remove` 、 `wait` 、 `signal` 四类操作，其中 `wait` 在资源不可用时阻塞当前进程，`signal` 在存在等待者时唤醒一个阻塞进程。

在用户态测试中，我实现了：

- `counter`：使用 `SpinLock` 和 `Semaphore` 分别保护计数器临界区。
- `mq`：使用 `EMPTY`、`FULL`、`MUTEX` 三个信号量实现生产者-消费者消息队列。
- `dinner`：使用筷子信号量与服务生信号量模拟并解决哲学家就餐问题。
- `fish`：创建三个子进程，分别只能输出 `>`、`<`、`_`，通过信号量保证输出总是 `<><_` 与 `><>_` 的组合。

四. 实验原理

1. `fork` 的进程复制模型

本实验中的 `fork` 更接近“共享地址空间的新执行流”。父子进程共享页表中的大部分用户映射，因此全局变量、静态变量和堆对象是共享的；但子进程需要独立栈空间，否则父子进程的局部变量和返回地址会互相覆盖。

实现时需要完成三件事：

1. 保存父进程当前上下文，使父进程之后能从系统调用返回。
2. 构造子进程 PCB，复制父进程上下文，并为子进程分配新的栈空间。
3. 设置不同返回值：父进程的 `rax` 为子进程 PID，子进程的 `rax` 为 0。

在我的实现中，`proc::fork` 先调用 `manager.save_current(context)` 保存父进程现场，再调用 `manager.fork()` 创建子进程，随后将父子进程都加入就绪队列，最后切换到下一个可运行进程：

```
pub fn fork(context: &mut ProcessContext) {
    x86_64::instructions::interrupts::without_interrupts(|| {
        let manager = get_process_manager();
        let parent_pid = manager.current().pid();

        manager.save_current(context);

        let child_pid = manager.fork();

        manager.push_ready(child_pid);
        manager.push_ready(parent_pid);

        manager.switch_next(context);
    })
}
```

`Process::fork` 负责创建新的 PID，调用 `ProcessInner::fork` 复制内部状态，并将子进程加入父进程的 `children` 列表。父进程返回值通过 `inner.context.set_rax(child_pid.0 as usize)` 设置，子进程返回值在 `ProcessInner::fork` 中设置为 0。

2. 子进程栈空间重定位

由于父子进程共享地址空间，如果子进程继续使用父进程栈地址，会导致两个执行流读写同一段栈。因此子进程需要一段新的用户栈。

实现中根据父子 PID 计算 `stack_offset_count`：

```
let stack_offset_count = u16::from(child_pid) as u64 - u16::from(self.pid) as u64;
```

随后在 `ProcessInner::fork` 中调用 `ProcessVm::fork(stack_offset_count)`，复制进程虚拟内存结构并为子进程栈选择偏移位置。之后根据旧栈所在范围调用 `context.relocate_stack` 调整子进程上下文中的栈指针，使其恢复执行时使用新栈：

```
let old_rsp = context.stack_frame.stack_pointer.as_u64();
let old_stack_start = old_rsp & STACK_START_MASK;
let old_stack_end = old_stack_start + STACK_MAX_SIZE;
context.relocate_stack(old_stack_start, old_stack_end, stack_offset);
context.set_rax(0);
```

这样，父子进程共享全局数据，但局部变量位于不同栈空间中，满足实验中对 `fork` 行为的要求。

3. 阻塞、唤醒与等待队列

轮询等待会持续占用 CPU 时间。例如父进程不断查询子进程是否退出，在子进程真正退出前，这些查询都没有实际工作。因此实验引入等待队列，使等待事件的进程主动进入 `Blocked` 状态。

进程状态中增加了 `Blocked`：

```
pub enum ProgramStatus {
    Running,
    Ready,
    Blocked,
    Dead,
}
```

`ProcessManager` 中维护等待队列：

```
wait_queue: Mutex<HashMap<ProcessId, BTreeSet<ProcessId>, ahash::RandomState>>,
```

其含义是：键为被等待进程 PID，值为正在等待该进程退出的一组进程 PID。阻塞时，当前进程状态变为 `Blocked`，并调用 `switch_next` 切换到其他就绪进程；唤醒时，将被唤醒进程状态改回 `Ready`，并放入就绪队列。

信号量也复用了这一思想。`wait` 发现资源数为 0 时，不忙等，而是把当前进程放入信号量的等待队列；`signal` 发现有等待进程时，唤醒其中一个进程。

4. 自旋锁

自旋锁用于用户态中较短的临界区。我的 `SpinLock` 基于 `AtomicBool` 实现：

```
pub fn acquire(&self) {
    loop {
        if self.bolt
            .compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed)
            .is_ok()
        {
            break;
        }

        while self.bolt.load(Ordering::Relaxed) {
            core::hint::spin_loop();
        }
    }
}

pub fn release(&self) {
    self.bolt.store(false, Ordering::Release);
}
```

`compare_exchange(false, true, ...)` 保证“检查锁是否空闲”和“占有锁”是一个不可分割的原子操作。若获取失败，则使用 `spin_loop` 提示 CPU 当前处于自旋等待。释放时用 `Release` 顺序写回 `false`，保证临界区内的写入在释放锁前完成。

5. 信号量

信号量由资源计数和等待队列组成：

```
pub struct Semaphore {
    count: usize,
    wait_queue: VecDeque<ProcessId>,
}
```

`wait` 的语义为：

- 若 `count > 0`，说明资源可用，减少计数并返回 `Ok`。
- 若 `count == 0`，说明资源不可用，把当前进程加入等待队列并返回 `Block(pid)`。

`signal` 的语义为：

- 若等待队列非空，取出一个等待进程并返回 `WakeUp(pid)`。
- 若没有等待进程，则增加资源计数。

内核系统调用根据 `SemaphoreResult` 操作调度器。`sem_wait` 遇到 `Block(pid)` 时保存当前上下文、阻塞当前进程并切换；`sem_signal` 遇到 `WakeUp(pid)` 时设置其返回值、改为 `Ready` 并加入就绪队列。

用户态库中将信号量封装为：

```
pub struct Semaphore {
    key: u32,
}

impl Semaphore {
    pub const fn new(key: u32) -> Self { Self { key } }
    pub fn init(&self, value: usize) -> bool { sys_new_sem(self.key, value) }
    pub fn remove(&self) -> bool { sys_remove_sem(self.key) }
    pub fn wait(&self) -> bool { sys_sem_wait(self.key) }
    pub fn signal(&self) -> bool { sys_sem_signal(self.key) }
}
```

这样用户程序不需要关心系统调用参数，只需要使用 `init/wait/signal/remove` 即可完成同步。

五. 实验过程

1. 系统调用扩展

在 `crates/syscall/src/lib.rs` 中加入 `Fork` 和 `Sem` 等系统调用号，并在用户态库中封装 `sys_fork`、`sys_sem_wait`、`sys_sem_signal`、`sys_new_sem`、`sys_remove_sem` 等函数。

系统调用分发函数根据 `rax` 中的系统调用号执行对应服务：

```
match args.syscall {
    Syscall::GetPid => context.set_rax(sys_getpid()),
    Syscall::Fork => sys_fork(context),
    Syscall::Exit => sys_exit(&args, context),
    Syscall::WaitPid => context.set_rax(sys_waitpid(&args) as usize),
    Syscall::Sem => sys_sem(&args, context),
    _ => { /* ... */ }
}
```

其中 `Fork` 和 `Sem::wait` 都可能导致上下文切换，因此服务函数需要拿到可变的 `ProcessContext`。

2. 进程管理结构扩展

`Process` 中维护：

- `pid`：进程号。
- `parent`：父进程弱引用。
- `children`：子进程列表。
- `status`：运行状态。
- `context`：寄存器和中断栈帧。
- `exit_code`：退出码。
- `proc_data`：进程共享资源，包括环境变量、文件资源和信号量集合。
- `proc_vm`：进程虚拟内存。

`ProcessManager` 维护进程表、就绪队列、应用列表和等待队列：

```
pub struct ProcessManager {
    processes: RwLock<HashMap<ProcessId, Arc<Process>, ahash::RandomState>>,
    ready_queue: Mutex<VecDeque<ProcessId>>,
    app_list: Option<boot::AppList>,
    wait_queue: Mutex<HashMap<ProcessId, BTreeSet<ProcessId>, ahash::RandomState>>,
}
```

调度器只会从就绪队列中选择状态为 `Ready` 的进程。阻塞进程即使仍然存在于进程表中，也不会被调度执行。

3. 用户态同步库

用户态同步库位于 `crates/lib/src/sync.rs`。自旋锁用于纯用户态忙等待，信号量通过系统调用进入内核，能够真正阻塞当前进程。

为了便于创建一组信号量，实现了 `semaphore_array!` 宏：

```
static CHOPSTICK: [Semaphore; 5] = semaphore_array![0, 1, 2, 3, 4];
```

这在哲学家就餐问题中用于快速声明五根筷子对应的信号量。

六. 用户程序测试

1. 多线程计数器

`counter` 程序创建 8 个并发执行流，每个执行流累加 100 次共享计数器。 `inc_counter` 内部刻意插入 `delay`，使读取、修改、写回三个步骤更容易被调度打断，从而暴露竞态条件。

未加锁时，两个执行流可能同时读取同一个旧值。例如两个进程都读到 `COUNTER = 10`，随后分别写回 `11`，最终丢失一次加法。使用锁后，任意时刻只有一个执行流进入 `inc_counter`，最终结果稳定为 800。

自旋锁版本：

```
fn do_counter_inc_spin() {  
    for _ in 0..100 {  
        SPIN_LOCK.acquire();  
        inc_counter();  
        SPIN_LOCK.release();  
    }  
}
```

信号量版本：

```
fn do_counter_inc_semaphore() {  
    for _ in 0..100 {  
        SEMAPHORE.wait();  
        inc_counter();  
        SEMAPHORE.signal();  
    }  
}
```

运行结果中应观察到：

```
SpinLock counter: 800  
Semaphore counter: 800
```

这说明两种同步机制均能正确保护临界区。

2. 消息队列

`mq` 程序实现生产者-消费者模型。父进程创建 16 个子进程，其中 8 个生产者、8 个消费者；每个生产者写入 10 条消息，每个消费者读取 10 条消息。队列容量分别测试 1, 4, 8, 16。

使用三个信号量：

- `MUTEX`：二元信号量，保护队列结构和输出信息。
- `EMPTY`：记录空槽数量，初值为队列容量。
- `FULL`：记录已有消息数量，初值为 0。

生产者逻辑：

```
EMPTY.wait();
MUTEX.wait();
push_message(msg);
MUTEX.signal();
FULL.signal();
```

消费者逻辑：

```
FULL.wait();
MUTEX.wait();
let msg = pop_message();
MUTEX.signal();
EMPTY.signal();
```

当队列满时，`EMPTY == 0`，生产者在 `EMPTY.wait()` 中阻塞；当队列空时，`FULL == 0`，消费者在 `FULL.wait()` 中阻塞。`MUTEX` 保证 `HEAD`、`TAIL`、`COUNT` 的修改是互斥的。

测试观察：

1. 队列容量为 1 时，生产者和消费者交替最明显。
2. 队列容量增大后，可以观察到多个生产者连续写入或多个消费者连续读取。
3. 每次测试结束后 `final queue count = 0`，说明生产和消费总量一致。
4. 输出中 `count = x/capacity` 始终满足 `0 <= x <= capacity`，说明队列没有越界写入或空读。

3. 哲学家就餐

`dinner` 程序创建 5 个并发执行流，每个哲学家循环进行“思考、拿筷子、吃饭、放筷子”。五根筷子分别由五个二元信号量保护：

```
static CHOPSTICK: [Semaphore; 5] = semaphore_array![
    0x2000, 0x2001, 0x2002, 0x2003, 0x2004
];
```

为了模拟不同情况，我设置了三种模式：

- `MODE = 0`：解决死锁版本。

- `MODE = 1` : 故意构造死锁。
- `MODE = 2` : 尝试观察饥饿或机会不均等。

死锁构造方式为：所有哲学家都先拿左筷子，再等待一段时间后尝试拿右筷子。此时可能出现五个哲学家各自持有一根筷子，并同时等待右侧筷子的情形，形成环路等待。

解决方式为引入“服务生”信号量 `ROOM`，初值为 `N - 1`：

```
static ROOM: Semaphore = Semaphore::new(0x2010);
assert!(ROOM.init(N - 1));
```

哲学家在拿筷子之前必须先进入 `ROOM`：

```
ROOM.wait();
// 再尝试拿左右筷子
```

由于最多只有 4 个哲学家同时竞争筷子，不可能出现 5 人同时各持有 1 根筷子的环路等待，因此破坏了死锁的必要条件之一。吃完后释放两根筷子并 `ROOM.signal()`，允许下一个哲学家进入竞争。

此外，程序使用基于 `pid`、哲学家编号和轮次的简单伪随机延迟：

```
fn random_delay(id: usize, round: usize, salt: usize) -> usize {
    let mut x = sys_get_pid() as usize;
    x ^= id * 1103515245;
    x ^= round * 12345;
    x ^= salt * 2654435761;
    x = x.wrapping_mul(1664525).wrapping_add(1013904223);
    x % 10 + 1
}
```

由于本实验环境中未实现 `sys_time`，这里使用 `sys_get_pid` 和循环变量作为种子来源，模拟不同执行流的竞争延迟。

实验现象：

- 在正常模式下，可以观察到不同哲学家输出 `eating round`，说明其同时获得左右筷子。
- 在死锁模式下，常见输出为多个哲学家均输出 `got left chopstick` 后停止，说明所有人都在等待右筷子。
- 在饥饿观察模式下，通过增加某个哲学家的思考延迟，可以观察其吃饭机会明显少于其他哲学家。

4. 加分项：fish 同步输出

`fish` 程序创建三个子进程：

- `gt_worker`：只能输出 `>`。

- `lt_worker` : 只能输出 `<` 。
- `under_worker` : 只能输出 `_` 。

三个子进程分别等待自己的信号量：

```
static SEM_GT: Semaphore = Semaphore::new(0x3000);
static SEM_LT: Semaphore = Semaphore::new(0x3001);
static SEM_UNDER: Semaphore = Semaphore::new(0x3002);
static SEM_DONE: Semaphore = Semaphore::new(0x3003);
```

父进程负责安排输出顺序。每次只唤醒一个子进程，并等待该子进程通过 `SEM_DONE` 告知打印完成：

```
fn print_one(sem: &Semaphore) {
    sem.signal();
    SEM_DONE.wait();
}
```

输出 `<><_` 的函数为：

```
fn print_group_lt_gt_lt() {
    print_one(&SEM_LT);
    print_one(&SEM_GT);
    print_one(&SEM_LT);
    print_one(&SEM_UNDER);
}
```

输出 `><>_` 的函数为：

```
fn print_group_gt_lt_gt() {
    print_one(&SEM_GT);
    print_one(&SEM_LT);
    print_one(&SEM_GT);
    print_one(&SEM_UNDER);
}
```

主循环交替调用这两个函数，因此输出总是 `<><_` 和 `><>_` 的组合。由于父进程在每个字符输出后等待 `SEM_DONE`，下一个字符不会提前输出；由于每个子进程代码中只调用 `put_char` 输出固定字符串，也满足“分别只能输出 `>`、`<` 和 `_`”的要求。

七. 编译与运行结果展示

1. counter 运行结果:

```

yyos> counter
[TRACE] [crates/kernel/src/proc/process.rs:73] New process counter#3 created.
[TRACE] [crates/elf/src/lib.rs:80] Loading ELF file... (user_access=true)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100000000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100001000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100004000) with flags PageTableFlags(PRESENT
| WRITABLE | USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100006000) with flags PageTableFlags(PRESENT
| WRITABLE | USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:52] Page Range: PageRange { start: Page[4KiB](0x3ffcffff000), end: Page[4KiB]
(0x3ffd00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:89] Init stack frame: InterruptStackFrame {
    instruction_pointer: VirtAddr(
        0x1111000013f0,
    ),
    code_segment: SegmentSelector {
        index: 5,
        rpl: Ring3,
    },
    cpu_flags: RFlags(
        IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
    ),
    stack_pointer: VirtAddr(
        0x3ffcfffff8,
    ),
    stack_segment: SegmentSelector {
        index: 6,
        rpl: Ring3,
    },
}
[TRACE] [crates/kernel/src/proc/manager.rs:291] New Process {
    pid: 3,
    name: "counter",
    parent: Some(
        2,
    ),
    status: Ready,
    ticks_passed: 0,
    children: Map {
        iter: Iter(
            [],
        ),
    },
}

```

```

status: Ready,
context: StackFrame {
  stack_top: VirtAddr(
    0x3ffcffffff8,
  ),
  cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
  ),
  instruction_pointer: VirtAddr(
    0x1111000013f0,
  ),
  regs: Registers
  r15: 0x0000000000000000, r14: 0x0000000000000000, r13: 0x0000000000000000,
  r12: 0x0000000000000000, r11: 0x0000000000000000, r10: 0x0000000000000000,
  r9 : 0x0000000000000000, r8 : 0x0000000000000000, rdi: 0x0000000000000000,
  rsi: 0x0000000000000000, rdx: 0x0000000000000000, rcx: 0x0000000000000000,
  rbx: 0x0000000000000000, rax: 0x0000000000000000, rbp: 0x0000000000000000,
},
vm: Some(
  ProcessVm {
    stack: Stack {
      top: 0x3ffd00000000,
      bot: 0x3ffcffffff00,
    },
    memory_usage: "4 KiB",
    page_table: PageTable {
      addr: PhysFrame[4KiB](0x173000),
      flags: Cr3Flags(
        0x0,
      ),
    },
  },
),
}
[DEBUG] [crates/kernel/src/proc/mod.rs:169] Spawned process: counter#3
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child counter_forked#4 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child counter_forked#5 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4]
Running 'counter' (PID: 3)...
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child counter_forked#6 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4, 5]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child counter_forked#7 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4, 5, 6]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child counter_forked#8 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4, 5, 6, 7]

```



```

[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child counter_forked#19 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 15]
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#12 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#13 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#14 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#15 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#16 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#17 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#18 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter_forked#19 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter_forked killed with exit code 0
Semaphore counter: 800
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x1234>
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process counter#3 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process counter killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process counter killed with exit code 0
[TRACE] [crates/kernel/src/proc/paging.rs:30] Releasing page table at PhysFrame[4KiB](0x173000)
Process 3 exited with code 0.
yyos>
yyos>

```

可看到，两种上锁的机制都正确保护了临界区的数据，最后输出结果都是预期的 800。

2. mq 运行结果

```

===== mq test capacity = 4 =====
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#37 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1]
[producer 0 pid 37] push msg 0, count = 1/4
[producer 0 pid 37] push msg 1, count = 2/4

```

```

[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#38 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 37]
[producer 0 pid 37] push msg 2, count = 3/4
[producer 1 pid 38] push msg 1000, count = 4/4
[producer 1 pid 38] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#39 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 37, 38]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#40 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 37]
[producer 0 pid 37] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#41 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 39]
[producer 2 pid 39] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#42 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 40]
[producer 3 pid 40] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#43 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 41]
[producer 4 pid 41] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#44 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 42]
[producer 5 pid 42] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#45 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 43]
[producer 6 pid 43] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#46 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 44]
[producer 7 pid 44] queue full, waiting; count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#47 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 45]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#48 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 46]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#49 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 45]
[consumer 0 pid 45] pop msg 0, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#50 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 46]
[consumer 1 pid 46] pop msg 1, count = 2/4
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#51 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 48, 37, 46]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child mq_forked#52 from parent #20
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 45]

```


parent created 16 processes

PID | PPID | Process Name | Ticks | Status

```
# 49 | # 20 | mq_forked | 1 | Ready
# 20 | # 2 | mq | 297 | Running
# 1 | # 0 | kernel | 115243 | Ready
# 37 | # 20 | mq_forked | 5 | Blocked
# 2 | # 1 | shell | 115239 | Ready
# 41 | # 20 | mq_forked | 2 | Blocked
# 42 | # 20 | mq_forked | 2 | Blocked
# 46 | # 20 | mq_forked | 4 | Blocked
# 51 | # 20 | mq_forked | 1 | Blocked
# 48 | # 20 | mq_forked | 2 | Blocked
# 43 | # 20 | mq_forked | 2 | Blocked
# 44 | # 20 | mq_forked | 2 | Blocked
# 47 | # 20 | mq_forked | 2 | Blocked
# 50 | # 20 | mq_forked | 1 | Blocked
# 52 | # 20 | mq_forked | 1 | Blocked
# 40 | # 20 | mq_forked | 2 | Blocked
# 45 | # 20 | mq_forked | 4 | Blocked
# 38 | # 20 | mq_forked | 3 | Blocked
# 39 | # 20 | mq_forked | 2 | Blocked
```

Heap : 25.297 KiB used / 31.975 MiB free / 32.000 MiB total

Queue : [2, 1, 49]

CPUs : [0: 20]

```
[consumer 2 pid 47] pop msg 2, count = 1/4
[producer 1 pid 38] push msg 1001, count = 2/4
[consumer 3 pid 48] pop msg 1000, count = 1/4
[producer 0 pid 37] push msg 3, count = 2/4
[producer 2 pid 39] push msg 2000, count = 3/4
[consumer 0 pid 45] pop msg 1001, count = 2/4
[producer 3 pid 40] push msg 3000, count = 3/4
[consumer 4 pid 49] pop msg 3, count = 2/4
[consumer 5 pid 50] pop msg 2000, count = 1/4
[producer 4 pid 41] push msg 4000, count = 2/4
[consumer 1 pid 46] pop msg 3000, count = 1/4
[producer 5 pid 42] push msg 5000, count = 2/4
[producer 6 pid 43] push msg 6000, count = 3/4
[consumer 6 pid 51] pop msg 4000, count = 2/4
[producer 7 pid 44] push msg 7000, count = 3/4
[consumer 7 pid 52] pop msg 5000, count = 2/4
[consumer 2 pid 47] pop msg 6000, count = 1/4
[producer 1 pid 38] push msg 1002, count = 2/4
[consumer 3 pid 48] pop msg 7000, count = 1/4
[producer 0 pid 37] push msg 4, count = 2/4
[producer 2 pid 39] push msg 2001, count = 3/4
[consumer 0 pid 45] pop msg 1002, count = 2/4
```

[producer 3 pid 40] push msg 3001, count = 3/4
[consumer 5 pid 50] pop msg 4, count = 2/4
[consumer 4 pid 49] pop msg 2001, count = 1/4
[producer 4 pid 41] push msg 4001, count = 2/4
[consumer 1 pid 46] pop msg 3001, count = 1/4
[producer 5 pid 42] push msg 5001, count = 2/4
[producer 6 pid 43] push msg 6001, count = 3/4
[consumer 6 pid 51] pop msg 4001, count = 2/4
[producer 7 pid 44] push msg 7001, count = 3/4
[consumer 7 pid 52] pop msg 5001, count = 2/4
[consumer 2 pid 47] pop msg 6001, count = 1/4
[producer 1 pid 38] push msg 1003, count = 2/4
[consumer 3 pid 48] pop msg 7001, count = 1/4
[producer 0 pid 37] push msg 5, count = 2/4
[producer 2 pid 39] push msg 2002, count = 3/4
[consumer 0 pid 45] pop msg 1003, count = 2/4
[producer 3 pid 40] push msg 3002, count = 3/4
[consumer 5 pid 50] pop msg 5, count = 2/4
[consumer 4 pid 49] pop msg 2002, count = 1/4
[producer 4 pid 41] push msg 4002, count = 2/4
[consumer 1 pid 46] pop msg 3002, count = 1/4
[producer 5 pid 42] push msg 5002, count = 2/4
[producer 6 pid 43] push msg 6002, count = 3/4
[consumer 6 pid 51] pop msg 4002, count = 2/4
[producer 7 pid 44] push msg 7002, count = 3/4
[consumer 7 pid 52] pop msg 5002, count = 2/4
[consumer 2 pid 47] pop msg 6002, count = 1/4
[producer 1 pid 38] push msg 1004, count = 2/4
[consumer 3 pid 48] pop msg 7002, count = 1/4
[producer 0 pid 37] push msg 6, count = 2/4
[producer 2 pid 39] push msg 2003, count = 3/4
[consumer 0 pid 45] pop msg 1004, count = 2/4
[producer 3 pid 40] push msg 3003, count = 3/4
[consumer 5 pid 50] pop msg 6, count = 2/4
[consumer 4 pid 49] pop msg 2003, count = 1/4
[producer 4 pid 41] push msg 4003, count = 2/4
[consumer 1 pid 46] pop msg 3003, count = 1/4
[producer 6 pid 43] push msg 6003, count = 2/4
[producer 5 pid 42] push msg 5003, count = 3/4
[consumer 6 pid 51] pop msg 4003, count = 2/4
[producer 7 pid 44] push msg 7003, count = 3/4
[consumer 7 pid 52] pop msg 6003, count = 2/4
[consumer 2 pid 47] pop msg 5003, count = 1/4
[producer 1 pid 38] push msg 1005, count = 2/4
[consumer 3 pid 48] pop msg 7003, count = 1/4
[producer 2 pid 39] push msg 2004, count = 2/4

```

[producer 0 pid 37] push msg 7, count = 3/4
[consumer 0 pid 45] pop msg 1005, count = 2/4
[producer 3 pid 40] push msg 3004, count = 3/4
[consumer 4 pid 49] pop msg 2004, count = 2/4
[consumer 5 pid 50] pop msg 7, count = 1/4
[producer 4 pid 41] push msg 4004, count = 2/4
[consumer 1 pid 46] pop msg 3004, count = 1/4
[producer 6 pid 43] push msg 6004, count = 2/4
[producer 5 pid 42] push msg 5004, count = 3/4
[consumer 6 pid 51] pop msg 4004, count = 2/4
[producer 7 pid 44] push msg 7004, count = 3/4
[consumer 7 pid 52] pop msg 6004, count = 2/4
[consumer 2 pid 47] pop msg 5004, count = 1/4
[consumer 3 pid 48] pop msg 7004, count = 0/4
[producer 1 pid 38] push msg 1006, count = 1/4
[producer 2 pid 39] push msg 2005, count = 2/4
[producer 0 pid 37] push msg 8, count = 3/4
[producer 3 pid 40] push msg 3005, count = 4/4
[consumer 0 pid 45] pop msg 1006, count = 3/4
[consumer 4 pid 49] pop msg 2005, count = 2/4
[consumer 5 pid 50] pop msg 8, count = 1/4
[consumer 1 pid 46] pop msg 3005, count = 0/4
[producer 4 pid 41] push msg 4005, count = 1/4
[producer 6 pid 43] push msg 6005, count = 2/4
[producer 5 pid 42] push msg 5005, count = 3/4
[producer 7 pid 44] push msg 7005, count = 4/4
[consumer 6 pid 51] pop msg 4005, count = 3/4
[consumer 7 pid 52] pop msg 6005, count = 2/4
[consumer 2 pid 47] pop msg 5005, count = 1/4
[consumer 3 pid 48] pop msg 7005, count = 0/4
[producer 1 pid 38] push msg 1007, count = 1/4
[producer 2 pid 39] push msg 2006, count = 2/4
[producer 0 pid 37] push msg 9, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#37 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 0 pid 45] pop msg 1007, count = 2/4
[producer 3 pid 40] push msg 3006, count = 3/4
[consumer 4 pid 49] pop msg 2006, count = 2/4
[consumer 5 pid 50] pop msg 9, count = 1/4
[producer 4 pid 41] push msg 4006, count = 2/4
[consumer 1 pid 46] pop msg 3006, count = 1/4
[producer 6 pid 43] push msg 6006, count = 2/4
[producer 5 pid 42] push msg 5006, count = 3/4
[consumer 6 pid 51] pop msg 4006, count = 2/4
[producer 7 pid 44] push msg 7006, count = 3/4

```

```

[consumer 7 pid 52] pop msg 6006, count = 2/4
[consumer 2 pid 47] pop msg 5006, count = 1/4
[producer 1 pid 38] push msg 1008, count = 2/4
[consumer 3 pid 48] pop msg 7006, count = 1/4
[producer 2 pid 39] push msg 2007, count = 2/4
[producer 3 pid 40] push msg 3007, count = 3/4
[consumer 0 pid 45] pop msg 1008, count = 2/4
[producer 4 pid 41] push msg 4007, count = 3/4
[consumer 4 pid 49] pop msg 2007, count = 2/4
[consumer 5 pid 50] pop msg 3007, count = 1/4
[producer 6 pid 43] push msg 6007, count = 2/4
[consumer 1 pid 46] pop msg 4007, count = 1/4
[producer 5 pid 42] push msg 5007, count = 2/4
[producer 7 pid 44] push msg 7007, count = 3/4
[consumer 6 pid 51] pop msg 6007, count = 2/4
[producer 1 pid 38] push msg 1009, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#38 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 7 pid 52] pop msg 5007, count = 2/4
[consumer 2 pid 47] pop msg 7007, count = 1/4
[producer 2 pid 39] push msg 2008, count = 2/4
[consumer 3 pid 48] pop msg 1009, count = 1/4
[producer 3 pid 40] push msg 3008, count = 2/4
[producer 4 pid 41] push msg 4008, count = 3/4
[consumer 0 pid 45] pop msg 2008, count = 2/4
[producer 6 pid 43] push msg 6008, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#45 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 4 pid 49] pop msg 3008, count = 2/4
[consumer 5 pid 50] pop msg 4008, count = 1/4
[producer 5 pid 42] push msg 5008, count = 2/4
[consumer 1 pid 46] pop msg 6008, count = 1/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#46 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[producer 7 pid 44] push msg 7008, count = 2/4
[producer 2 pid 39] push msg 2009, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#39 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 6 pid 51] pop msg 5008, count = 2/4
[producer 3 pid 40] push msg 3009, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#40 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0

```

```

[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 2 pid 47] pop msg 7008, count = 2/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#47 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 7 pid 52] pop msg 2009, count = 1/4
[producer 4 pid 41] push msg 4009, count = 2/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#41 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 3 pid 48] pop msg 3009, count = 1/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#48 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[producer 6 pid 43] push msg 6009, count = 2/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#43 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[producer 5 pid 42] push msg 5009, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#42 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[producer 7 pid 44] push msg 7009, count = 4/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#44 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 4 pid 49] pop msg 4009, count = 3/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#49 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 5 pid 50] pop msg 6009, count = 2/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#50 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 6 pid 51] pop msg 5009, count = 1/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#51 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
[consumer 7 pid 52] pop msg 7009, count = 0/4
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process mq_forked#52 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process mq_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process mq_forked killed with exit code 0
mq capacity 4 finished, final queue count = 0
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x1001>
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x1002>
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x1003>

```

```
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x1001>1
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x1002>8
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x1003>0
```

3. dinner 运行结果

```
yyos> dinner
[TRACE] [crates/kernel/src/proc/process.rs:73] New process dinner#3 created.
[TRACE] [crates/elf/src/lib.rs:80] Loading ELF file... (user_access=true)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100000000) with flags PageTableFlags(PRESENT | USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100001000) with flags PageTableFlags(PRESENT | USER_ACCESSIBLE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100004000) with flags PageTableFlags(PRESENT | WRITABLE | USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:52] Page Range: PageRange { start: Page[4KiB](0x3ffcffff000), end: Page[4KiB](0x3ffd00000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:89] Init stack frame: InterruptStackFrame {
  instruction_pointer: VirtAddr(
    0x111100001880,
  ),
  code_segment: SegmentSelector {
    index: 5,
    rpl: Ring3,
  },
  cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
  ),
  stack_pointer: VirtAddr(
    0x3ffcfffff8,
  ),
  stack_segment: SegmentSelector {
    index: 6,
    rpl: Ring3,
  },
}
[TRACE] [crates/kernel/src/proc/manager.rs:291] New Process {
  pid: 3,
  name: "dinner",
  parent: Some(
    2,
  ),
}
```

```

status: Ready,
ticks_passed: 0,
children: Map {
  iter: Iter(
    [],
  ),
},
status: Ready,
context: StackFrame {
  stack_top: VirtAddr(
    0x3ffcffff8,
  ),
  cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
  ),
  instruction_pointer: VirtAddr(
    0x111100001880,
  ),
  regs: Registers
  r15: 0x0000000000000000, r14: 0x0000000000000000, r13: 0x0000000000000000,
  r12: 0x0000000000000000, r11: 0x0000000000000000, r10: 0x0000000000000000,
  r9: 0x0000000000000000, r8: 0x0000000000000000, rdi: 0x0000000000000000,
  rsi: 0x0000000000000000, rdx: 0x0000000000000000, rcx: 0x0000000000000000,
  rbx: 0x0000000000000000, rax: 0x0000000000000000, rbp: 0x0000000000000000,
},
vm: Some(
  ProcessVm {
    stack: Stack {
      top: 0x3ffd00000000,
      bot: 0x3ffcffff000,
    },
    memory_usage: "4 KiB",
    page_table: PageTable {
      addr: PhysFrame[4KiB](0x173000),
      flags: Cr3Flags(
        0x0,
      ),
    },
  },
),
}
[DEBUG] [crates/kernel/src/proc/mod.rs:169] Spawned process: dinner#3
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x2000>1
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x2001>1
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x2002>1
Running 'dinner' (PID: 3)...

```

```

[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x2003>1
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x2004>1
[TRACE] [crates/kernel/src/proc/sync.rs:87] Sem Insert: <0x2010>4
dining philosophers start, mode = 0
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child dinner_forked#4 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child dinner_forked#5 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child dinner_forked#6 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4, 5]
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child dinner_forked#7 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4, 5, 6]
[P0 pid 4] thinking round 0
[DEBUG] [crates/kernel/src/proc/process.rs:116] Forked child dinner_forked#8 from parent #3
[TRACE] [crates/kernel/src/proc/manager.rs:77] Ready queue: [2, 1, 4, 5, 6, 7]
[P0] got left chopstick 0
[P1 pid 5] thinking round 0
[P1] got right chopstick 2
[P2 pid 6] thinking round 0
[P3 pid 7] thinking round 0
[P3] got right chopstick 4
[P4 pid 8] thinking round 0
[P0] got right chopstick 1
[P0 pid 4] eating round 0, delay = 9
[P3] got left chopstick 3
[P3 pid 7] eating round 0, delay = 9
[P0] put chopsticks 0 and 1
[P1] got left chopstick 1
[P1 pid 5] eating round 0, delay = 9
[P3] put chopsticks 3 and 4
[P4] got left chopstick 4
[P4] got right chopstick 0
[P4 pid 8] eating round 0, delay = 9
[P0 pid 4] thinking round 1
[P1] put chopsticks 1 and 2
[P3 pid 7] thinking round 1
[P2] got left chopstick 2
[P2] got right chopstick 3
[P2 pid 6] eating round 0, delay = 9
[P4] put chopsticks 4 and 0
[P3] got right chopstick 4
[P0] got left chopstick 0
[P1 pid 5] thinking round 1
[P4 pid 8] thinking round 1
[P0] got right chopstick 1
[P0 pid 4] eating round 1, delay = 4

```


[P2] put chopsticks 2 and 3
 [P3] got left chopstick 3
 [P3 pid 7] eating round 1, delay = 4
 [P1] got right chopstick 2
 [P0] put chopsticks 0 and 1
 [P3] put chopsticks 3 and 4
 [P4] got left chopstick 4
 [P0 pid 4] thinking round 2
 [P3 pid 7] thinking round 2
 [P0] got left chopstick 0
 [P2 pid 6] thinking round 1
 [P1] got left chopstick 1
 [P1 pid 5] eating round 1, delay = 4
 [P1] put chopsticks 1 and 2
 [P0] got right chopstick 1
 [P0 pid 4] eating round 2, delay = 9
 [P2] got left chopstick 2
 [P1 pid 5] thinking round 2
 [P2] got right chopstick 3
 [P2 pid 6] eating round 1, delay = 4
 [P0] put chopsticks 0 and 1
 [P4] got right chopstick 0
 [P4 pid 8] eating round 1, delay = 4
 [P2] put chopsticks 2 and 3
 [P1] got right chopstick 2
 [P4] put chopsticks 4 and 0
 [P3] got right chopstick 4
 [P1] got left chopstick 1
 [P1 pid 5] eating round 2, delay = 9
 [P2 pid 6] thinking round 2
 [P4 pid 8] thinking round 2
 [P0 pid 4] thinking round 3
 [P3] got left chopstick 3
 [P3 pid 7] eating round 2, delay = 9
 [P1] put chopsticks 1 and 2
 [P2] got left chopstick 2
 [P0] got left chopstick 0
 [P3] put chopsticks 3 and 4
 [P4] got left chopstick 4
 [P2] got right chopstick 3
 [P2 pid 6] eating round 2, delay = 9
 [P1 pid 5] thinking round 3
 [P0] got right chopstick 1
 [P0 pid 4] eating round 3, delay = 4
 [P3 pid 7] thinking round 3
 [P0] put chopsticks 0 and 1

[P2] put chopsticks 2 and 3
[P4] got right chopstick 0
[P4 pid 8] eating round 2, delay = 9
[P1] got right chopstick 2
[P0 pid 4] thinking round 4
[P2 pid 6] thinking round 3
[P3] got right chopstick 4
[P0] got left chopstick 0
[P4] put chopsticks 4 and 0
[P1] got left chopstick 1
[P1 pid 5] eating round 3, delay = 4
[P1] put chopsticks 1 and 2
[P0] got right chopstick 1
[P0 pid 4] eating round 4, delay = 9
[P2] got left chopstick 2
[P3] got left chopstick 3
[P3 pid 7] eating round 3, delay = 4
[P4 pid 8] thinking round 3
[P1 pid 5] thinking round 4
[P3] put chopsticks 3 and 4
[P4] got left chopstick 4
[P2] got right chopstick 3
[P2 pid 6] eating round 3, delay = 4
[P0] put chopsticks 0 and 1
[P3 pid 7] thinking round 4
[P2] put chopsticks 2 and 3
[P1] got right chopstick 2
[P4] got right chopstick 0
[P4 pid 8] eating round 3, delay = 4
[P2 pid 6] thinking round 4
[P1] got left chopstick 1
[P1 pid 5] eating round 4, delay = 9
[P0 pid 4] thinking round 5
[P4] put chopsticks 4 and 0
[P3] got right chopstick 4
[P0] got left chopstick 0
[P4 pid 8] thinking round 4
[P3] got left chopstick 3
[P3 pid 7] eating round 4, delay = 9
[P1] put chopsticks 1 and 2
[P2] got left chopstick 2
[P0] got right chopstick 1
[P0 pid 4] eating round 5, delay = 4
[P3] put chopsticks 3 and 4
[P0] put chopsticks 0 and 1
[P2] got right chopstick 3

```

[P2 pid 6] eating round 4, delay = 9
[P4] got left chopstick 4
[P0 pid 4] thinking round 6
[P0] got left chopstick 0
[P1 pid 5] thinking round 5
[P0] got right chopstick 1
[P0 pid 4] eating round 6, delay = 9
[P2] put chopsticks 2 and 3
[P3 pid 7] thinking round 5
[P1] got right chopstick 2
[P0] put chopsticks 0 and 1
[P4] got right chopstick 0
[P4 pid 8] eating round 4, delay = 9
[P2 pid 6] thinking round 5
[P1] got left chopstick 1
[P1 pid 5] eating round 5, delay = 4
[P0 pid 4] thinking round 7
[P1] put chopsticks 1 and 2
[P2] got left chopstick 2
[P4] put chopsticks 4 and 0
[P3] got right chopstick 4
[P0] got left chopstick 0
[P1 pid 5] thinking round 6
[P4 pid 8] thinking round 5
[P2] got right chopstick 3
[P2 pid 6] eating round 5, delay = 4
[P0] got right chopstick 1
[P0 pid 4] eating round 7, delay = 4
[P2] put chopsticks 2 and 3
[P1] got right chopstick 2
[P3] got left chopstick 3
[P3 pid 7] eating round 5, delay = 4
[P0] put chopsticks 0 and 1
[P0 pid 4] done
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process dinner_forked#4 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process dinner_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process dinner_forked killed with exit code 0
[P2 pid 6] thinking round 6
[P1] got left chopstick 1
[P1 pid 5] eating round 6, delay = 9
[P3] put chopsticks 3 and 4
[P4] got left chopstick 4
[P3 pid 7] thinking round 6
[P1] put chopsticks 1 and 2
[P4] got right chopstick 0
[P4 pid 8] eating round 5, delay = 4

```

```

[P2] got left chopstick 2
[P4] put chopsticks 4 and 0
[P2] got right chopstick 3
[P2 pid 6] eating round 6, delay = 9
[P3] got right chopstick 4
[P4 pid 8] thinking round 6
[P1 pid 5] thinking round 7
[P2] put chopsticks 2 and 3
[P1] got right chopstick 2
[P3] got left chopstick 3
[P3 pid 7] eating round 6, delay = 9
[P2 pid 6] thinking round 7
[P3] put chopsticks 3 and 4
[P1] got left chopstick 1
[P1 pid 5] eating round 7, delay = 4
[P4] got left chopstick 4
[P4] got right chopstick 0
[P4 pid 8] eating round 6, delay = 9
[P1] put chopsticks 1 and 2
[P1 pid 5] done
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process dinner_forked#5 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process dinner_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process dinner_forked killed with exit code 0
[P2] got left chopstick 2
[P3 pid 7] thinking round 7
[P2] got right chopstick 3
[P2 pid 6] eating round 7, delay = 4
[P4] put chopsticks 4 and 0
[P3] got right chopstick 4
[P2] put chopsticks 2 and 3
[P2 pid 6] done
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process dinner_forked#6 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process dinner_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process dinner_forked killed with exit code 0
[P4 pid 8] thinking round 7
[P3] got left chopstick 3
[P3 pid 7] eating round 7, delay = 4
[P3] put chopsticks 3 and 4
[P3 pid 7] done
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process dinner_forked#7 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process dinner_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process dinner_forked killed with exit code 0
[P4] got left chopstick 4
[P4] got right chopstick 0
[P4 pid 8] eating round 7, delay = 4
[P4] put chopsticks 4 and 0

```

```

[P4 pid 8] done
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process dinner_forked#8 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process dinner_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process dinner_forked killed with exit code 0
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x2000>
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x2001>
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x2002>
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x2003>
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x2004>
[TRACE] [crates/kernel/src/proc/sync.rs:92] Sem Remove: <0x2010>
dining philosophers finished
[DEBUG] [crates/kernel/src/proc/process.rs:85] Killing process dinner#3 with ret code: 0
[TRACE] [crates/kernel/src/proc/process.rs:211] Process dinner killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process dinner killed with exit code 0
[TRACE] [crates/kernel/src/proc/paging.rs:30] Releasing page table at PhysFrame[4KiB](0x173000)
Process 3 exited with code 0.
yyos>

```

这个问题为了避免死锁，最多只允许四个哲学家同时进食，不难看出程序正常运行。

4. fish 运行结果

```

yyos> fish
[TRACE] [crates/kernel/src/proc/process.rs:73] New process fish#9 created.
[TRACE] [crates/elf/src/lib.rs:80] Loading ELF file... (user_access=true)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100000000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100001000) with flags PageTableFlags(PRESENT
| USER_ACCESSIBLE)
[TRACE] [crates/elf/src/lib.rs:121] Mapping segment at VirtAddr(0x111100004000) with flags PageTableFlags(PRESENT
| WRITABLE | USER_ACCESSIBLE | NO_EXECUTE)
[TRACE] [crates/elf/src/lib.rs:52] Page Range: PageRange { start: Page[4KiB](0x3ff6ffff000), end: Page[4KiB]
(0x3ff700000000) }(1)
[TRACE] [crates/kernel/src/proc/context.rs:89] Init stack frame: InterruptStackFrame {
  instruction_pointer: VirtAddr(
    0x111100001730,
  ),
  code_segment: SegmentSelector {
    index: 5,
    rpl: Ring3,
  },
}

```

```

cpu_flags: RFlags(
    IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
),
stack_pointer: VirtAddr(
    0x3ff6ffffff8,
),
stack_segment: SegmentSelector {
    index: 6,
    rpl: Ring3,
},
}
[TRACE] [crates/kernel/src/proc/manager.rs:291] New Process {
    pid: 9,
    name: "fish",
    parent: Some(
        2,
    ),
    status: Ready,
    ticks_passed: 0,
    children: Map {
        iter: Iter(
            [],
        ),
    },
    status: Ready,
    context: StackFrame {
        stack_top: VirtAddr(
            0x3ff6ffffff8,
        ),
        cpu_flags: RFlags(
            IOPL_HIGH | IOPL_LOW | INTERRUPT_FLAG,
        ),
        instruction_pointer: VirtAddr(
            0x111100001730,
        ),
        regs: Registers
        r15: 0x0000000000000000, r14: 0x0000000000000000, r13: 0x0000000000000000,
        r12: 0x0000000000000000, r11: 0x0000000000000000, r10: 0x0000000000000000,
        r9 : 0x0000000000000000, r8 : 0x0000000000000000, rdi: 0x0000000000000000,
        rsi: 0x0000000000000000, rdx: 0x0000000000000000, rcx: 0x0000000000000000,
        rbx: 0x0000000000000000, rax: 0x0000000000000000, rbp: 0x0000000000000000,
    },
}

```



```
[TRACE] [crates/kernel/src/proc/process.rs:211] Process fish killed with exit code 0
[TRACE] [crates/kernel/src/proc/process.rs:219] Process fish killed with exit code 0
[TRACE] [crates/kernel/src/proc/paging.rs:30] Releasing page table at PhysFrame[4KiB](0x173000)
Process 9 exited with code 0.
yyos>
```

八. 思考题

1. 1. 输入缓冲区使用 **Mutex** 保护时需要注意什么？

如果在串口输入中断和用户态读取之间使用同一个 **Mutex** 保护输入队列，需要避免在持锁期间发生可能再次获取同一把锁的中断。典型问题是：内核正在执行 **pop**，已经获得输入队列锁，但尚未释放；此时串口中断到来，中断处理程序尝试 **push** 新字符，也要获取同一把锁。由于当前 CPU 已经持有该锁，中断处理程序会自旋等待，而原来的 **pop** 又无法继续执行并释放锁，从而造成死锁。

解决方法是：在访问该缓冲区的临界区内关闭中断，保证持锁期间不会被同类中断打断；或者使用无锁队列，使中断上下文中的 **push** 不依赖可能被普通上下文持有的睡眠锁或自旋锁。若使用锁，还应保证临界区尽可能短，不在持锁期间执行阻塞或耗时操作。

2. 2. **fork** 复制内存时各页表之间是什么关系？

系统当前页表是 CPU 正在使用的页表，通常对应当前运行进程。进程页表描述该进程的虚拟地址空间。子进程页表由父进程页表派生而来，本实验中共享主要用户映射和内核映射，但为子进程单独安排栈空间。内核页表提供内核代码、数据和物理内存映射，用户进程页表通常由内核页表的高地址部分克隆或共享，使中断和系统调用进入内核后仍能访问内核空间。

复制内存时要注意：

1. 不能在正在使用的页表和目标页表之间造成地址别名冲突。
2. 子进程栈必须映射到独立区域，并复制父进程当前栈内容。
3. 修改子进程上下文中的 **rsp**，否则子进程仍会使用父进程栈地址。
4. 父子进程返回值不同，需要分别设置各自上下文中的 **rax**。
5. 若共享页表或物理页，需要使用引用计数管理生命周期，避免一个进程退出时释放仍被另一个进程使用的页表资源。

3.3. 为什么 fork 必须在堆分配前进行？

本实验的 `fork` 不实现完整地址空间复制，也没有实现写时复制。父子进程会共享堆和全局数据。如果在堆分配之后 `fork`，父子进程会共享同一个用户态堆分配器状态。之后任一进程执行 `alloc` 或 `dealloc` 都可能修改同一套堆元数据，导致重复分配、重复释放、链表损坏或悬垂指针等问题。

此外，Rust 的很多抽象会隐式使用堆，例如 `Vec`、`String`、格式化输出中的临时分配等。如果在这些分配之后 `fork`，父子进程共享的堆状态可能与各自执行路径不一致，出现难以定位的内存错误。因此实验要求在任何 Rust 堆分配前调用 `fork`。

4.4. Ordering 参数的含义

`Ordering` 用于描述原子操作与其他内存访问之间的顺序约束。常见取值如下：

- `Relaxed`：只保证该原子变量本身操作的原子性，不建立额外的跨线程顺序关系。
- `Acquire`：常用于加锁或读取同步标志，保证该操作之后的读写不会被重排到它之前。
- `Release`：常用于解锁或发布同步标志，保证该操作之前的读写不会被重排到它之后。
- `AcqRel`：同时具有 `Acquire` 和 `Release` 语义，常用于读-改-写操作。
- `SeqCst`：顺序一致性，是最强的内存顺序，所有线程看到的 `SeqCst` 原子操作顺序一致。

在本实验的自旋锁中，获取锁使用 `Acquire`，释放锁使用 `Release`。这足以保证进入临界区后能看到前一个持锁者释放前完成的写入。

5.5. 为什么 SpinLock 需要实现 Sync？Send 又是什么？

`Sync` 表示一个类型的共享引用 `&T` 可以安全地在线程之间共享。静态锁变量会被多个并发执行流通过共享引用访问，因此 `SpinLock` 需要实现 `Sync`。虽然 `SpinLock` 内部有可变状态，但该状态由 `AtomicBool` 以原子方式访问，满足并发安全要求，所以可以手动 `unsafe impl Sync for SpinLock`。

`Send` 表示一个类型的所有权可以安全地从一个线程转移到另一个线程。简单地说，`Send` 关注“值能不能被移动到别的线程”，`Sync` 关注“共享引用能不能被多个线程同时访问”。对于锁类型，通常更关心 `Sync`，因为多个线程共享的是同一个锁对象。

6.6. spin_loop 的 pause 和 hlt 有什么区别？为什么不能用 hlt？

`core::hint::spin_loop` 在 x86_64 上通常会生成 `pause` 指令。`pause` 是给处理器的提示，表示当前处于短暂自旋等待，可以降低流水线和超线程资源竞争，但 CPU 仍然继续执行当前线程。

`hlt` 则会让 CPU 进入停止状态，直到下一次外部中断到来。它通常用于内核空闲循环，而不是用户态锁等待。

自旋锁不能使用 `hlt`，原因是锁的释放不一定伴随硬件中断。如果等待者执行 `hlt` 后没有中断到来，它可能无法及时继续检查锁状态。用户态程序也不应该直接执行这类特权/低层电源管理语义的指令。因此自旋等待使用 `pause` 更合适。

九. 实验总结

本次实验将前几次实验中的进程、调度、系统调用和用户态程序串联起来。`fork` 的实现让我理解到，进程复制不只是创建一个 PCB，还必须正确处理上下文、返回值、父子关系和栈空间。阻塞与唤醒机制则体现了调度器的核心价值：等待资源时不应浪费 CPU，而应让出处理器给其他就绪进程。

在并发测试中，计数器展示了竞态条件的直观后果；消息队列展示了信号量对资源数量和临界区的组合控制；哲学家就餐问题展示了死锁、饥饿以及通过破坏环路等待解决死锁的思路；

`fish` 程序则验证了信号量不仅能保护资源，也能精确控制多个进程的执行顺序。

总体而言，本实验使我对“并发不是同时写代码，而是精确设计等待关系”有了更具体的认识。同步原语本身并不复杂，真正需要谨慎的是它们与调度、进程状态和共享资源生命周期之间的配合。